

3장 데스크톱 버전 검토 의견

대상 문서: 노션 「3장 클로드 코드 시작하기 (손글씨 인식 앱) 데스크톱 버전」

검토 관점: 2026년 9월부터 수업을 듣는 대학 1학년이 혼자 따라 할 때 어디서 막히고 무엇을 오해하는가

검토 범위: 본문 전체와 삽입된 그림 115장 전부

읽는 법

심각도는 학생이 실제로 막히는가를 기준으로 매겼다.

등급	뜻
A	여기서 멈춘다. 다음 단계로 못 간다
B	진행은 되지만 틀리게 이해하거나, 화면이 달라 불안해한다
C	다듬으면 좋다

A 등급 5건, B 등급 7건, C 등급 5건이다. A 등급 다섯 건 중 넷이 3.1 설치와 3.2 첫 실습에 몰려 있다. 첫 90분에서 절반이 이탈할 위험이 있다.

A 등급 — 학생이 여기서 멈춘다

A-1. Homebrew 를 설치하는 단계가 없다

절 제목이 "(Homebrew 활용한) 맥 설치 방법"인데, 본문은 터미널 열기 → `brew install --cask claude` 로 바로 넘어간다.

Homebrew 는 맥에 기본으로 깔려 있지 않다. 1학년 대부분의 맥에는 없다. 그 학생이 보는 화면은 이것이다.

```
zsh: command not found: brew
```

문서 어디에도 이 문장이 없고, 무엇을 해야 하는지도 없다. 첫 명령에서 수업이 멈춘다.

고칠 것 — 설치 절 맨 앞에 확인과 설치를 넣는다. 아래는 노션에 그대로 옮겨 쓸 수 있는 문안이다.

0. Homebrew 가 있는지 먼저 확인한다

터미널에 아래를 입력한다.

```
brew --version
```

Homebrew 4.x.x 처럼 버전이 나오면 다음으로 넘어간다.

command not found: brew 가 나오면 아래를 실행해 먼저 설치한다. 5~10분 걸리고, 중간에 맥 로그인 암호를 묻는다. 암호는 화면에 표시되지 않는다. 그냥 입력하고 엔터.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

설치가 끝나면 터미널이 Next steps: 로 시작하는 안내를 준다. M칩 맥이면 거기에 아래 세 줄이 표시된다. 같은 터미널 창에 그대로 복사해 실행해야 brew 명령을 쓸 수 있다. /Users/사용자이름 자리에는 각자의 계정 이름이 이미 채워져 나온다.

```
echo >> /Users/사용자이름/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> /Users/사용자이름/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

앞의 두 줄이 설정 파일에 기록하고, 세 번째 줄이 지금 창에 즉시 적용한다.

인텔 맥은 이 단계가 없다. 설치 위치(/usr/local/bin)가 이미 PATH 에 있어 Next steps: 에 PATH 안내 자체가 안 나온다. 내 칩은 애플 메뉴 → 이 Mac에 관하여 에서 확인한다.

마지막으로 터미널을 새로 열고 brew --version 을 다시 확인한다.

윈도우도 같은 문제다

winget 은 윈도우 11 에는 기본 탑재지만, 윈도우 10 에는 없거나 너무 낮은 경우가 많다. 확인 단계와 설치 방법을 함께 넣는다.

```
winget --version
```

v1.x.x 처럼 버전이 나오면 넘어간다. winget '은(는) 내부 또는 외부 명령... 이 아닙니다' 가 나오면 설치해야 한다.

방법 1 — 마이크로소프트 스토어 (권장). 1학년에게는 이쪽만 안내해도 충분하다.

시작 → Microsoft Store 를 열고 앱 설치 관리자 로 검색한다. (영문 환경이면 App Installer)
받기 또는 업데이트 를 누른다. 설치가 끝나면 PowerShell 을 새로 열고 winget --version 을 다시 확인한다.

winget 은 독립 프로그램이 아니라 앱 설치 관리자 라는 앱에 들어 있다. 이름이 달라서 스토어에서 "winget" 으로 검색하면 안 나온다. 학생이 가장 많이 헤매는 지점이다.

방법 2 — 스토어를 못 쓰는 경우. 학교 관리 PC 에서 스토어가 막혀 있으면 PowerShell 로 직접 설치한다.

```
Invoke-WebRequest -Uri https://aka.ms/getwinget -OutFile winget.msixbundle  
Add-AppxPackage winget.msixbundle
```

이 방법은 의존 패키지(VCLibs 등)가 없으면 실패할 수 있다. 그때는 오류 메시지를 클로드에게 그대로 붙여 넣으면 필요한 것을 알려 준다.

윈도우 10 이 아주 낡았다면 winget 자체가 안 깔린다. 윈도우 10 버전 1809(빌드 17763) 이상이 필요하다. 설정
→ 시스템 → 정보 에서 확인한다. 이 경우 윈도우를 먼저 업데이트해야 한다.

A-2. gh 명령을 설치하는 단계가 없다

3.3 깃허브 올리지의 첫 단계가 gh auth login 이다. GitHub CLI 도 맥에 기본으로 없다. A-1 과 완전히 같은 실패가 반복된다.

고칠 것 — 그 앞에 한 줄 넣는다.

```
brew install gh
```

(윈도우는 winget install --id GitHub.cli)

A-3. 실습 결과가 환경에 따라 달라지는데, 그 사실이 문서에 없다

이 문서에서 가장 값이 큰 지적이다.

첫 프롬프트에 대한 클로드의 응답(그림 48)에 이렇게 적혀 있다.

Python 3.14 뿐이고 TensorFlow 는 3.14용 휠이 없어 설치 불가입니다. (책 예제는 보통 Keras 를 쓰지만 이 환경에선 안 됩니다.) PyTorch 2.13 은 설치 가능하므로 PyTorch 로 CNN 을 학습하겠습니다.

즉 이 문서의 모든 후속 화면은 "파이썬 3.14 가 깔린 교수님 맥"이라는 우연한 조건의 산물이다. model.py , train.py , preprocess.py , app.py 라는 파일 구성도, PyTorch 라는 선택도 마찬가지다.

파이썬 3.12 를 쓰는 학생에게 클로드는 십중팔구 Keras 로 짤다. 그러면,

- 만들어지는 파일 이름이 다르다
- 가중치 파일이 `mnist_cnn.pt` 가 아니다
- 이후 3.3 의 웹 버전 실습(가중치내보내기.py 가 `.pt` 를 읽는다)이 통째로 어긋난다

학생은 자기가 뭘 잘못했다고 생각한다. 실제로는 아무것도 잘못하지 않았다.

한 가지 확인이 필요하다. 위 인용에서 클로드가 "책 예제는 보통 Keras 를 쓰지만" 이라고 단정하는데, 이 강의 교재의 예제가 실제로 Keras 인지는 이 검토에서 확인하지 못했다. 만약 교재가 Keras 를 쓴다면 학생은 교재와 실습이 서로 다른 프레임워크를 쓰는 상태가 되므로, 그 사실을 수업에서 먼저 밝혀야 한다. 교재가 프레임워크를 지정하지 않는다면 이 항목은 신경 쓰지 않아도 된다. 어느 쪽이든 아래 처방은 그대로 유효하다.

고칠 것 — 3.2 시작 부분에 실습 전제를 명시한다. 아래는 그대로 옮겨 쓸 수 있는 문안이다.

시작하기 전에 — 내 파이썬 버전을 확인한다

```
python3 -V
```

이 문서의 화면은 파이썬 3.14 환경에서 만들어졌다. 그 환경에서는 TensorFlow 가 설치되지 않아 클로드가 PyTorch 를 골랐고, 그래서 `mnist_cnn.pt` 라는 파일이 나온다.

여러분의 버전이 3.12 나 3.13 이면 클로드는 Keras 를 고를 수 있다. 그러면 파일 이름과 구조가 이 문서의 화면과 달라진다. 그것은 틀린 것이 아니다. 다만 3.3 의 웹 버전 실습은 PyTorch 결과물을 전제로 하므로, 수업에서는 아래 프롬프트로 PyTorch 를 지정한다.

손글씨로 숫자를 입력하면 이것을 인식하는 코드를 만들어서 실행해 줘.
PyTorch 를 사용하고, 학습된 가중치는 `mnist_cnn.pt` 로 저장해 줘.
모든 코드와 주석을 한글로 작성해 줘.

이것이 이 수업에서 배우는 핵심 중 하나다. 같은 프롬프트라도 환경이 다르면 AI 는 다른 답을 낸다. 결과를 그대로 믿지 말고 무엇을 골랐는지 읽어야 한다.

마지막 문단이 특히 중요하다. 약점을 교육 소재로 바꾸는 지점이다.

A-4. SSL 인증서 오류가 스크린샷 안에만 있다

같은 화면(그림 48, 49)에 이런 내용이 있다.

SSL 인증서 문제입니다. 파이썬 설치를 건드리지 않고, MNIST 원본 파일을 `curl` 로 내려받아 놓겠습니다.

근본 해결을 원하시면 `/Applications/Python 3.14/Install Certificates.command` 를 한 번 실행하시면 됩니다.

맥에 파이썬 공식 설치본을 쓰면 아주 흔하게 터지는 오류다. 학생 상당수가 만난다. 그런데 본문에는 한 줄도 없고, 축소된 스크린샷 안에서만 읽힌다.

고칠 것 — 본문 박스로 끌어올린다. 아래는 그대로 옮겨 쓸 수 있는 문안이다.

MNIST 를 내려받다가 **SSL: CERTIFICATE_VERIFY_FAILED** 가 뜨면

맥의 파이썬이 인증서를 못 읽어서 그렇다. 파인더에서 응용 프로그램 > Python 3.14 폴더를 열고 **Install Certificates.command** 를 더블클릭한 뒤 실습을 다시 시작한다. (3.14 자리는 `python3 -V` 로 확인한 자기 버전으로 읽는다.)

클로드에게 "SSL 인증서 오류가 났어" 라고 말해도 우회 방법을 찾아 준다.

A-5. 3.4 핵심 정리가 이 문서의 내용과 맞지 않는다

핵심 키워드 8개(그림 94) 중 셋이 이 문서에 없거나 문서와 모순된다.

키워드	정리의 설명	실제 이 문서
[2] 노드JS / 깃	"실행 환경(노드JS)과 버전 관리 도구(깃)"	노드JS 를 쓰지 않는다. 데스크톱 앱은 Homebrew 로 깔았다. 노드JS 는 CLI 버전(<code>npm install -g</code>) 이야기다
[4] 배치 파일	"윈도우 명령어를 모아 실행하는 .bat 파일"	배치 파일이 문서에 한 번도 안 나온다
[7] # 키	"CLAUDE.md에 규칙 빠르게 추가 하는 단축키"	없어진 기능이다. 이 문서 그림 74 가 직접 "다만 현재 문서에는 없습니다" 라고 확인해 준다

CLI 버전 장의 정리를 그대로 가져온 것으로 보인다. 문서가 스스로 없어졌다고 밝힌 기능을 마지막 정리에서 다시 가르치고 있다. 학생은 시험에 나오는 줄 알고 외운다.

고칠 것 — 이 문서에서 실제로 한 것으로 교체한다. 예를 들어 이런 8개다.

#	키워드	한 줄
1	클로드 코드	AI 와 대화하며 코드를 만들고 고치는 도구
2	Homebrew / winget	맥, 윈도우에 프로그램을 명령 한 줄로 설치하는 도구

#	키워드	한 줄
3	MNIST	손글씨 숫자 7만 장의 표준 데이터셋
4	권한 모드	클로드에게 어디까지 말길지 정하는 설정 (수동 ~ 권한 무시)
5	CLAUDE.md	프로젝트 설정 파일. 세션 시작 시 자동 로드
6	/init	CLAUDE.md 를 자동 생성하는 명령
7	계층적 구조	루트에 공통, 하위 폴더에 특화 규칙을 나눠 관리
8	깃 / 깃허브 / Pages	기록(커밋) → 올리기(푸시) → 웹 주소로 공개(배포)

B 등급 — 진행은 되지만 오해하거나 불안해한다

B-1. 부동소수 오차 설명의 예시가 실제로는 오차가 나지 않는다

그림 90, 92 에 이렇게 적혀 있다.

면적 평균 축소에서 $20 * (180/20)$ 이 180.00000000000003 이 되는 부동소수 오차 때문에...

틀렸다. 실제로 확인하면 정확히 **180.0** 이다.

```
>>> 20 * (180/20)
180.0
```

호기심 있는 학생이 파이썬에 쳐 보면 오차가 안 난다. 그러면 설명 전체를 못 믿게 된다.

오차가 실제로 나는 값은 이것이다. 저장소(`web_version/CLAUDE.md`)에서는 이미 정정했지만 노션 문서에는 옛 설명이 남아 있다.

```
>>> 19 * (21/19)
21.000000000000004
```

고칠 것 — 잘라낸 너비 21 을 새 너비 19 로 줄이는 경우로 예시를 바꾼다. 이 값은 마지막 열에서 인덱스가 하나 넘쳐 실제로 `undefined` → `NaN` 을 만든다.

B-2. 저장소 이름이 본문과 화면에서 다르다

본문의 예시 프롬프트는 이렇다.

이 폴더를 깃허브 저장소로 만들어서 올려 줘. 저장소 이름은 my-first-app, 공개로.

그런데 실제 화면(그림 71)은 Study01_MNIST_mac 이고, 문서 뒷부분의 링크도 전부 그쪽이다.

그대로 따라 한 학생은 my-first-app 이라는 저장소를 만든다. 그러면 뒤에 나오는 배포 주소도, 최종 결과 링크도 자기 것과 안 맞는다.

고칠 것 — 실습 프롬프트를 실제 이름으로 통일한다.

이 폴더를 깃허브 저장소로 만들어서 올려 줘. 저장소 이름은 Study01_MNIST_mac, 공개로.

B-3. 권한 모드 표의 번호가 1, 2, 3, √, 5 다

네 번째 줄의 번호 칸에 4 대신 √ 가 들어 있다. 드롭다운(그림 45)에서 자동이 현재 선택돼 체크로 표시된 것을 그대로 옮긴 결과다.

학생은 "4번은 어디 갔지?" 하고 멈춘다.

고칠 것 — 번호 칸은 4 로 적고, 선택 여부는 별도 열이나 배경색으로 표시한다.

B-4. 학습에 몇 분 걸리는지 알려주지 않는다

CNN 5 에폭 학습은 맥에서 수 분 걸린다. 그동안 화면에 진행 표시가 거의 없다.

1학년은 멈춘 줄 알고 중단한다. 중단하면 mnist_cnn.pt 가 안 생기고, 이후 전부 실패한다.

고칠 것 — 한 줄 넣는다.

학습에 3~10분 걸린다. 화면이 멈춘 것처럼 보여도 끄지 않는다. 맥 사양에 따라 다르다.

B-5. "폴더에서 새로운 터미널 열기" 가 기본 메뉴가 아니다

그림 2 의 서비스 메뉴는 이미 활성화된 상태다. 맥 기본값은 꺼져 있다. 게다가 그림에는 반디집, 7z 항목도 함께 있어 학생 화면과 눈에 띄게 다르다.

고칠 것 — 활성화 방법을 덧붙이거나, 더 확실한 대안을 함께 준다.

이 메뉴가 안 보이면 시스템 설정 → 키보드 → 키보드 단축키 → 서비스 → 파일 및 폴더 에서
폴더에서 새로운 터미널 열기 를 켜다.

또는 터미널을 먼저 연 뒤 cd 를 입력하고 폴더를 터미널 창으로 끌어다 놓으면 경로가 자동으로 입력된다.

B-6. 삭제 → 설치 → 갱신 순서다

처음 설치하는 학생에게 "기존 앱 삭제 방법"이 맨 앞에 나온다. 지울 것이 없는 학생은 혼란스럽다.

고칠 것 — 설치를 맨 앞에 두고, 갱신과 삭제는 뒤에 "이미 깔려 있다면"이라는 조건과 함께 접어 둔다.

B-7. 3.1 예시 폴더와 3.2 실습 폴더가 다르다

3.1 코드 모드 설명은 3장 손글씨 앱 폴더, 3.2 실습은 study01_MNIST(Mac) 폴더다. 폴더를 언제 어디에 만드는지 학생 머릿속에서 꼬인다.

고칠 것 — 3.1에서도 study01_MNIST(Mac) 을 쓰거나, 3.1의 폴더는 "화면 구경용 예시"라고 밝힌다.

C 등급 — 다듬으면 좋다

C-1. 스킬을 화면에서 쓰는데 설명이 없다

그림 87 에 스킬 실행됨 /superpowers:writing-plans 가 그대로 찍혀 있다. 학생이 같은 프롬프트를 넣어도 그 스킬이 설치돼 있지 않으면 이 화면이 안 나온다.

설계 문서와 계획 문서가 만들어지는 과정 전체가 이 스킬 덕분인데, 문서는 그 존재를 언급하지 않는다.

고칠 것 — 별도로 작성한 [스킬 사용 현황과 설치 안내](#) 문서를 3.3 에서 링크한다.

C-2. 로그인 화면 그림이 맥 화면이 아니다

그림 9(맥 설치 절)의 로그인 창은 윈도우 창 장식(오른쪽 위 최소화/최대화/닫기)이다. 맥은 왼쪽 위 빨강, 노랑, 초록이다. 사소하지만 "내 화면과 다른데?" 를 한 번 더 만든다.

C-3. 정확도 수치가 문서마다 다르다

그림 49 는 "앱 예측 경로 정확도 98.0% (테스트 300장)", 그림 92 의 최종 검증표는 "98.0% (196/200)" 이다. 표본 수가 다르다. 어느 쪽이 기준인지 학생이 헷갈린다.

고칠 것 — 200장 기준으로 통일하거나, 둘이 다른 검증이라는 점을 밝힌다.

C-4. 화면 속 개인 사용 기록이 그대로 보인다

그림 19, 43 등에 세션 47개, 메시지 25,364개, 총 토큰 14.4M, 그리고 지난 세션 제목들이 보인다. 학생 화면은 텅 비어 있다. 지우거나, "교수님 화면이라 목록이 많다" 는 한 줄을 붙이면 좋다.

C-5. .app 더블클릭 설정은 다른 기기로 안 따라간다

그림 64 에 클로드가 직접 밝힌 내용이 있다.

이 설정은 `app.py` 파일 하나에만 붙는 확장 속성(xattr)이라, 파일을 다른 곳으로 복사하면 따라가지 않을 수 있습니다. SynologyDrive 동기화 폴더라 다른 기기에서는 다시 지정해야 할 수 있습니다.

학교 실습실과 집을 오가는 학생에게 실제로 일어나는 일이다. 본문에 짧게 옮겨 둘 가치가 있다.

잘 되어 있는 점

고칠 것만 적으면 균형이 안 맞으니, 이 문서가 이미 잘하고 있는 것도 적는다.

- `git init`, 커밋, 저장소 생성, 푸시를 사진첩 비유로 푼 대목이 아주 좋다. "커밋은 내 컴퓨터, 푸시해야 올라간다" 를 오해 표로 정리한 것도 1학년에게 정확히 필요한 처방이다.
- "토큰을 만들어 `~/.zshrc` 에 넣으라는 인터넷 안내는 따르지 마세요" 라는 경고가 들어 있다. 실제로 이 프로젝트에서 그 문제를 겪고 얻은 교훈이라 무게가 다르다.
- 9개 미흡한 점을 표로 공개하고 "검증 통과는 옳다가 아니라 시험해 본 범위 안에서는 옳다" 로 맺은 부분은 이 장 전체에서 교육적으로 가장 값이 크다. AI 가 만든 것을 그대로 믿지 않는 태도를 실제 사례로 가르친다.
- 배포가 느리니 취소하지 말라는 경고가 있다. 실제로 이 프로젝트가 그 함정에 빠졌던 내용이라 근거가 있다.

우선순위 제안

수업 시작까지 시간이 한정돼 있다면 이 순서를 권한다.

순서	항목	이유
1	A-1, A-2 (Homebrew, gh 설치 단계)	30분이면 고친다. 안 고치면 첫 시간에 절반이 막힌다
2	A-3 (환경 의존성 경고 + PyTorch 지정 프롬프트)	고치는 김에 이 수업의 핵심 메시지가 된다
3	A-5 (핵심 정리 교체)	학생이 외우는 내용이라 틀리면 손해가 크다
4	A-4, B-1, B-2	각각 10분 내외
5	나머지 B, C	학기 중에 순차 반영

검토 방법

- 노션 문서를 API 로 내려받아 본문 전체(이미지 주소를 뺀 32,578자)를 읽었다.
- 삽입된 그림 115장을 모두 내려받아 확인했다. 101장에는 캡션이 없어 그림을 보지 않으면 평가할 수 없는 상태였다.

- 본문의 사실 주장은 가능한 범위에서 직접 확인했다. 부동산수 예시(B-1)는 파이썬으로 실행해 대조했고, 정확도 수치는 저장소의 `검증.html` 을 실제로 돌려(순전파 최대 절대차 $2.68e-7$, 정확도 98.0%) 확인했다.
- Homebrew 의 **Next steps:** 세 줄은 이 맥(애플 실리콘)의 `~/.zprofile` 에 실제로 남은 결과(`eval "$(/opt/homebrew/bin/brew shellenv)"`)로 확인했다.

확인하지 못한 것도 밝힌다.

- 학생 맥에서 처음부터 따라 해 보지는 않았다. A-1, A-2 는 "기본으로 설치돼 있지 않다"는 사실에 근거한 추론이다.
- 윈도우 항목은 이 맥에서 실행해 볼 수 없었다. `winget` 설치 방법, 빌드 17763 요건, 앱 설치 관리자 라는 이름은 일반적으로 알려진 사실이고 직접 시험하지 않았다. 수업 전에 윈도우 PC 한 대에서 밟아 보기를 권한다.
- 교재를 확인하지 않았다. A-3 에 인용한 "책 예제는 보통 Keras 를 쓰지만" 은 그림 48 안에서 클로드가 한 말이고, 그 클로드가 무엇을 근거로 그렇게 말했는지도 확인되지 않았다. 교재 원문을 보고 판단할 사항이다.